August 2026, India

Version 1.1

Boyer-Moore Algorithm

Tutorial

Tutorial By

Vikas Awadhiya

This work is licensed under CC-BY 4.0

Boyer-Moore Algorithm Tutorial © 2026 by Vikas Awadhiya is licensed under Creative Commons Attribution 4.0 International (CC-BY 4.0).
To view a copy of this license, visit

https://creativecommons.org/licenses/by/4.0/

Invented By,

Robert S. Boyer

&

J Strother Moore

Tutorial By

Vikas Awadhiya

LinkedIn profile: https://in.linkedin.com/in/awadhiya-vikas

1 Introduction

Boyer-Moore algorithm is an efficient string searching algorithm. It searches the pattern in the string in linear time complexity in average case scenario. It has quadratic time complexity in worst case scenario but that is rare in practice. In best case scenario, Boyer-Moore algorithm performs better than linear time complexity. As compared to Boyer-Moore algorithm, naive (brute-force) algorithm has quadratic time complexity.

Boyer-Moore algorithm obtains the efficiency by changing the conventional approach of matching the pattern from left to right. The Boyer-Moore algorithm matches the pattern from right to left which means it begins matching from the last character of the pattern and moves backward to the first character.

Algorithm pre-processes the pattern to obtain information about it. This information is gathered in terms of two tables. These tables help to decide how many positions to the right the pattern must slides in case of a character mismatch. The naive algorithm, in case of a character mismatch, slides the pattern one position to the right and comparisons re-begins from first character of the pattern but Boyer-Moore algorithm tries to slide the pattern as far as possible to the right, which can be less than or equal to the length of pattern and after the pattern right sliding, the character matching begins again from the last character of the pattern.

The space complexity is close to linear, it requires two tables, the first table has size equal to the number of letters in the alphabet of a language or unique number of characters in the pattern if Hash Table data-structure is used and the second table has size equal to the length of the pattern.

2 Pattern Searching

Initially the string and patter left align and algorithm matches the characters of the pattern from right to left and begins from last character of the pattern. In case of a character mismatch naive algorithm simply moves the pattern one position to the right and matching begins again from first character of the patter but for the Boyer-Moore algorithm, a mismatch is not just a mismatch rather algorithm creates three categories of a character mismatch and these categories allow more efficiently patter sliding in case of a mismatch. The categories are as follows,

2.1 When A Mismatched Character Is Not Present In The Pattern

If the string **S** = **"abcabdcbabcbcaba"** and the pattern **Patt** = **"abcbca"** then,

S =	a	b	c	a	b	d	c	b	a	b	c	b	c	a	b	a
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Patt =	a	b	c	b	c	a										
Index =>	1	2	3	4	5	6										

Fig 2.1

As highlighted by the gray color filled cells above in fig-2.1, the last character of the pattern doesn't match with respective character 'd' of the string and mismatched character 'd' is also not present in the pattern. It means the pattern cannot slide to the right by distance less than pattern's length because no character in the pattern can match with character 'd'. So, the pattern slides six positions to the right beyond the character 'd' and character matching begins again from the last character of the pattern as shown below in fig-2.2,

S =	a	b	c	a	b	d	c	b	a	b	c	b	c	a	b	a
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Patt =							a	b	c	b	c	a				
Index =>							1	2	3	4	5	6				

Fig 2.2

In above example this mismatch scenario occurs at last character of the pattern but usually it can occur at any position in the pattern.

2.2 When A Mismatched Character Is Present In The Pattern

This is a specific scenario of a mismatch at the last character of the pattern. If we reconsider the previous string and pattern as shown above in fig-2.2. The last character of pattern doesn't match with respective character of the string but this time the mismatched character 'b' is present in the pattern. To avoid the further mismatch, the pattern must be slid right in such way that the character 'b' of the pattern aligns to the character 'b' of the string but here the character 'b' occurs twice in the pattern. So, if the pattern has multiple occurrences of a character, then rightmost occurrence of the character must be aligned. In this case the rightmost occurrence of the character 'b' is at two positions away from the last character of the pattern. So, the pattern slides two positions to the right and the character matching begins again from the last character of the pattern as shown below (in next page) in fig-2.3,

S =	a	b	c	a	b	d	c	b	a	b	c	b	c	a	b	a
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Patt =									a	b	c	b	c	a		
Index =>									1	2	3	4	5	6		

Fig 2.3

2.3 When A Mismatched Character Is Present In The Pattern But A Mismatch Is Not Occurred At The Last Character

In this scenario the mismatch character is present in the pattern and a mismatch occurs after some successful matches and it is not at the last character of the pattern.

If the string **S** = **"abacbbadabcabdcab"** and patter **Patt** = **"dabcabdcab"** then,

S =	a	b	a	c	b	b	a	d	a	b	c	a	b	d	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Patt =	d	a	b	c	a	b	d	c	a	b							
Index =>	1	2	3	4	5	6	7	8	9	10							

Fig 2.4

As highlighted by the gray color filled cells above in $\text{fig-}2.4$, a character mismatch occurres after two matches and it is not at the last character of the pattern.

The mismatched character 'd' is present in the pattern ($\text{Patt}[7]$) and the distance between the rightmost occurrence of the character 'd' and the mismatch position is one. So, the pattern can slide one position to the right but in this category there is an another possible way in which pattern can slide and algorithm selects the why which offers maximum pattern sliding.

As show above in the $\text{fig-}2.4$, the remaining pattern after the character 'c' ($\text{Patt}[8]$) in the pattern can be referred as a sub-pattern/suffix and if this sub-pattern, $\text{Patt}[9, 10] = \text{"ab"}$ recurs in the pattern and it is not preceded by the same character 'c' then pattern can slide right to align the recurrence of the sub-pattern/suffix.

Patt =	d	a	b	c	a	b	d	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10

Fig 2.5

As show above in the $\text{fig-}2.5$, all recurrences of the sub-pattern are highlighted by the gray color filled cells. There are two recurrences of sub-pattern but one of them is preceded by the same character 'c' and it cannot be used but the other recurrence close to the left-end of the patter is preceded by a different character 'd'. So, the pattern slides seven positions to the right which is equal to the distance between mismatch index 8 and the index 1 of preceded character 'd' of the sub-pattern recurrence. Here this way offers a higher number of positions sliding. The matching begins again from pattern's last character as highlighted by gray color cells below in $\text{fig-}2.6$.

S =	a	b	a	c	b	b	a	d	a	b	c	a	b	d	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Patt =								d	a	b	c	a	b	d	c	a	b
Index =>								1	2	3	4	5	6	7	8	9	10

Fig 2.6

3 Tables Creation

The Boyer-Moore algorithm creates delta_1 & delta_2 tables by the pattern pre-processing to look at all pattern sliding possibilities in case of a character mismatch.

As explained in previous section, in case of a mismatch, if the mismatched character is not present in pattern then there is no option but to slide pattern right beyond the mismatched character (first mismatch category). If a mismatch occurs at the last character of the pattern and if mismatched character present in the pattern then pattern slides right to align rightmost occurrence of a mismatched character (second mismatch category).

In case of third mismatch category when a mismatch doesn't occur at the last character of the pattern then algorithm has two options, either pattern can slide to align mismatched character or pattern can slide to align the recurrence of sub-pattern/suffix. Algorithm selects the option which offer maximum sliding.

So in case of a mismatch, delta_1 table offers a number of positions the pattern can slide to align the mismatched character and delta_2 table offers a number of positions the pattern can slide to align the recurrence of sub-pattern, algorithm selects maximum value among them.

The Boyer-Moore algorithm is available as a standard library in many programming languages. It is available in C++ from C++17 onward (`std::boyer_moore_searcher`). So it is not required to implement and that's why this document explains the algorithm with 1-based index.

3.1 Delta$_1$ Table

The table contains one entry for each letter of the alphabet and has size equal to the number of letters in the alphabet of a language (the real world implementation uses Hash-Table data-structure).

If a letter is not present in the patter, it is assumed, its entry in delta_1 table has pattern length, as a value. If a letter is present, its entry has a value equal to the difference between pattern length and index of its rightmost occurrence in the pattern. Let's consider a pattern, **Patt** = **abcdbabcab** of length 10 then its delta_1 table is as follows,

Patt =	a	b	c	d	b	a	b	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10

	a	b	c	d
Delta$_1$ =	1	0	2	6

Fig 3.1

The value of each entry of the delta_1 table is evaluated by following formula,

$$\begin{aligned}\mathbf{delta_1[character]} &= (\ \mathbf{mismatch\ index} - \mathbf{rightmost\ index\ of\ a\ character}\ ) + \\ &\quad (\ \mathbf{pattern\ length} - \mathbf{mismatch\ index}\ ) \\ &= \mathbf{pattern\ length} - \mathbf{rightmost\ index\ of\ a\ character}\end{aligned}$$

A positive and a negative instances of mismatch index variable cancel-out each other and the formula simplified to difference between pattern length and $\text{rightmost index of a character}$. This makes delta_1 table independent of mismatch index and allows delta_1 table to be evaluated even before the character matching begins.

The simplified version of delta_1 formula is good for calculation but the original formula with two sub-parts helps to understand the concept. So to understand the formula let's consider an example of a character mismatch as shown below in $\text{fig-}3.2$,

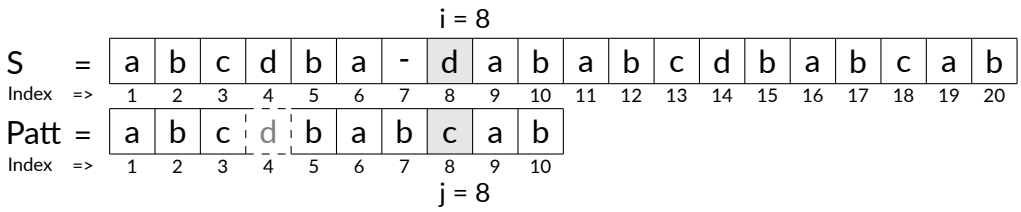

Fig 3.2

As highlighted by gray color filled cells above in $\text{fig-}3.2$, a mismatch occurs after two matches at $j = 8$ and the respective character 'd' of the string (at $i = 8$) is present in the pattern and it's rightmost occurrence is at index 4, as highlighted by dashed boundary column.

But this mismatch belongs to the third category of mismatch as explained in previous section and pattern can also slide three positions to the right, to align the recurrence of sub-pattern $\text{Patt}[9, 10] = \text{"ab"}$, which recurs at index 6 and it is preceded by difference character 'd' ($\text{Patt}[5]$) in the pattern.

Here the alignment of a mismatched character offers 4 positions sliding to the right as compared to the 3 positions sliding offers by the alignment of a sub-pattern recurrence, so algorithm selects maximum value and slides the pattern four positions to the right as shown below in $\text{fig-}3.3$,

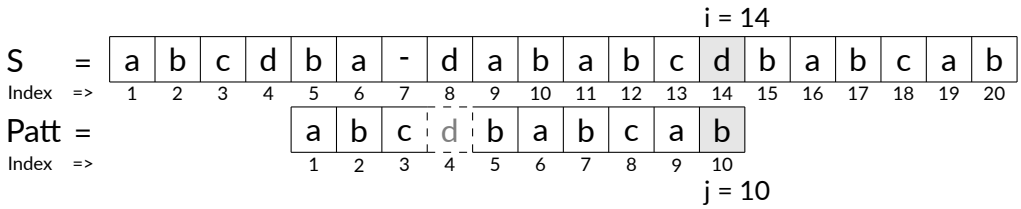

Fig 3.3

After patter sliding, character matching begins again from the last character of the pattern that's why variable j is set to $j = 10$. The variable i is set to $i = 14$ because last character of pattern align to the character at index 14 of the string, as highlighted by gray filled cells, above in $\text{fig-}3.3$. Obviously algorithm can't manually see the alignment rather it uses tables to update variable i.

This demonstrates, the pattern sliding at conceptual level turns into the variables update at implementation level. So the variable j is set to $j = \text{pattern length}$ after a character mismatch and the value of variable i is updated by the maximum value offered by tables.

A value of delta_1 (or delta_2 table) is not only the number of positions to slider rather it contains one additional factor as well which helps to update the variable i properly. So let's see both sub-parts of the delta_1 formula one by one.

The first sub-part of the delta_1 formula evaluates number of positions to slide. As shown above in $\text{fig-}3.2$, mismatch occurs at $j = 8$ and the rightmost occurrence of mismatched character 'd' is at index 4 in the pattern as highlighted by dashed boundary cell. That's why $\text{sub-part}_1 = \text{ mismatch index } - \text{ rightmost occurrence of a character } = 8 - 4 = 4$. So pattern slides four positions to the right, as shown below (in next page) in $\text{fig-}3.4$,

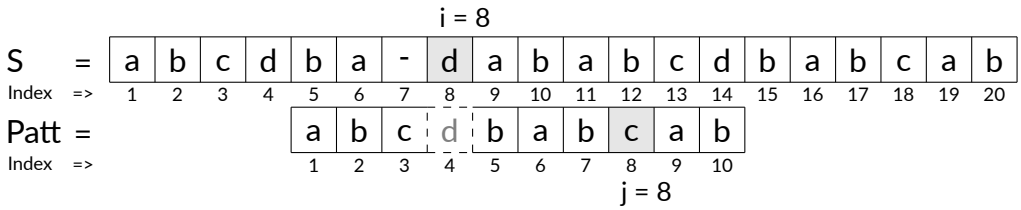

Fig 3.4

As shown above in fig-3.4, variable i is still at $i = 8$ but when a mismatch occurred i and j were aligned, hence i must be incremented (moved) by the same number of positions (four positions) as the pattern slid to the right, as shown below in fig-3.5,

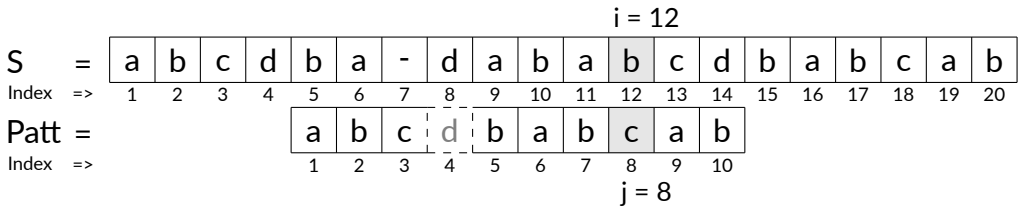

Fig 3.5

To begin the character matching again, j must be incremented to last index of pattern and the second sub-part of the formula evaluates how many positions j is away from last index by, $\text{sub-part}_2 = \text{pattern length} - \text{mismatch index} = 10 - 8 = 2$, hence variables i and j are incremented by two positions, as shown below in fig-3.6,

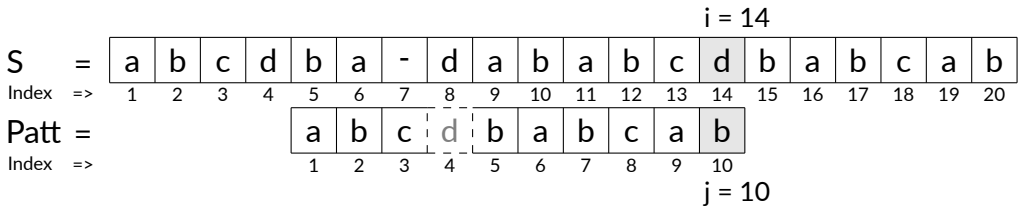

Fig 3.6

This is the reasoning of delta_1 formula and what the each entry of delta_1 table represents. So algorithm in this case simply uses delta_1 entry of mismatched character 'd' to update variable i to $i = i + \text{delta}_1['d'] = 8 + 6 = 14$ (delta_1 table shown above in fig-3.1) and set j to $j = \text{patter length} = 10$.

It is possible in case of a character mismatch that a rightmost occurrence of a mismatched character is at the right side of the mismatched index in the pattern, in such a case delta_2 table offers a relevant and bigger value.

3.2 Delta₂ Table

The delta_2 table provides rightmost recurrence information of the sub-patterns/suffixes. The number of entries in delta_2 table is equal to the pattern's length. The $\text{delta}_2[j]$ provides the recurrence information of a sub-pattern/suffix begins at the index $j + 1$.

To find the recurrence of a sub-pattern in the pattern is not the same problem as finding a suffix in the string. The Boyer-Moore algorithm has its own specific version of it.

As discussed the rightmost recurrence of a sub-pattern, $\text{Patt}[j + 1, \text{pattern length}]$ must not

preceded by the same character $\mathrm{patt}[j]$, but the algorithm allows the special / partial recurrence of the sub-pattern if the non-recurred part falls outside of the left-end of the pattern and in this way it differs from regular problem to find recurrence of a suffix. So let's consider a pattern **Patt = abcab**, as an example to understand it,

Index =>	1	2	3	4	5
Patt =	a	b	c	a	b

Fig 3.7

The sub-pattern/suffix, $\mathrm{Patt}[4,5] = \text{"ab"}$ is preceded by a character 'c' ($\mathrm{Patt}[3]$) as highlighted by gray color filled cell above in fig-3.7. The sub-pattern "ab" recurs at index 1 the left-end of the pattern. So there is no character to precede the recurrence of sub-pattern and it is equivalent to say the recurrence of the sub-pattern is not preceded by the same character 'c'.

But to maintain the similarity and to overcome a specific problem when there is no complete recurrence of sub-pattern, algorithm assumes a special character '$' and also assumes it doesn't appear in the pattern. This special character '$' at the same time can match or mismatch with any character. In addition, algorithm also assumes the presence of character '$' before index 1 as many times as it is requires in a particular case. So for the sub-pattern $\mathrm{Patt}[4,5]$, the pattern would be assumed as shown below in fig-3.8,

Index =>	0	1	2	3	4	5
Patt =	$	a	b	c	a	b
Sub-Patt =	c	a	b			

Fig 3.8

It is assumed, the character '$' at index 0 doesn't match with preceding character 'c' of the sub-pattern. So the sub-pattern recurs at index 1 and it is not preceded by the same character 'c'.

The sub-pattern, $\mathrm{Patt}[3,5] = \text{"cab"}$ is preceded by a character 'b' ($\mathrm{Patt}[2]$) and the pattern would be assumed as shown below in fig-3.9,

Index =>	-1	0	1	2	3	4	5
Patt =	$	$	a	b	c	a	b
Sub-Patt =	b	c	a	b			

Fig 3.9

The sub-Pattern, $\mathrm{Patt}[3,5]$ partially recurs in the pattern that's why, it is assumed, there are two instances of character '$' present before index 1 in the pattern. So the sub-pattern recurs at index 0, where it is assumed the character '$' ($\mathrm{Patt}[0]$) matches with the respective character 'c' of the sub-pattern and the character '$' ($\mathrm{patt}[-1]$) doesn't matches with the preceding character 'b' of the sub-pattern.

Finally the sub-pattern, $\mathrm{Patt}[2,5] = \text{"bcab"}$ is preceded by a character 'a' ($\mathrm{Patt}[1]$) and the pattern would be assumed as shown below in fig-3.10,

Index =>	-2	-1	0	1	2	3	4	5
Patt =	$	$	$	a	b	c	a	b
Sub-Patt =	a	b	c	a	b			

Fig 3.10

The sub-pattern, $\mathrm{Patt}[2,5]$ recurs at index -1 (yes a negative index is allowed) where it is assumed, the character '$' the $\mathrm{Patt}[0]$ and $\mathrm{Patt}[-1]$ match to the character 'c' and the character 'b' of the sub-pattern, respectively and the character '$' ($\mathrm{Patt}[-2]$) doesn't match with the preceding character 'a' of the sub-pattern.

This technique allows to assume the recurrence of a sub-pattern/suffix even when it doesn't recur or recurs practically. That's why the values of delta_2 table always remain relevant unlike the values of delta_1 table.

Now let's consider a pattern, **Patt** = **"abcdabcab"** and see how concept explained above is used to evaluate delta_2 table. The value of each entry of delta_2 table is evaluated by following formula,

$$\begin{aligned}\mathbf{delta_2}[j] &= (\ (j+1) - \textbf{sub-pattern recurrence index}\ ) + (\ \textbf{pattern length} - j\ )\\ &= (\ \textbf{pattern length} + 1) - \textbf{sub-pattern recurrence index}\end{aligned}$$

The non-simplified version of this formula has two sub-parts and these sub-parts have the same reasoning as the formula of delta_1 table has, which is already explained.

Let's see each value of $\mathrm{delta}_2[j]$, where the index j iterates from $j = \mathrm{pattern\ length}$, to $j = 1$. The current index j is highlighted by gray color filled cell.

Index =>	1	2	3	4	5	6	7	8	9
Patt =	a	b	c	d	a	b	c	a	b
Delta$_2$ =									1

Fig 3.11

At $j = 9$, as highlighted by the gray color filled cell above in fig-3.11, the last character of pattern, its delta_2 value will always be 1 because, at $j = 9$ there is no sub-pattern begins at $j + 1 = 10$, rather it is considered as an empty sub-pattern and due to this, the empty sub-pattern recurrence index is also index 9 and its delta_2 value is,

delta$_2$ [9] = (9 + 1) – sub-pattern recurrence index + (pattern length - 9) = (9 + 1) – 9 + (9 – 9) = 1

At $j = 8$ the sub-pattern begins at $j + 1 = 9$ and it is $\mathrm{Pattern}[9,9] = \text{"b"}$, but there is no recurrence of sub-pattern "b" in the pattern which is preceded by a character other than the character 'a'. So it is assumed the sub-pattern recurs at index 0 (beyond the left-end of the pattern) where the character '$' ($\mathrm{Patt}[0]$) matches with character 'b' but character '$' ($\mathrm{Patt}[-1]$) doesn't match with preceded character 'a' as shown below in fig-3.12 and its delta_2 value is,

delta$_2$ [8] = (8 + 1) – 0 + (9 – 8) = 10,

Index =>	-1	0	1	2	3	4	5	6	7	8	9
Patt =	$	$	a	b	c	d	a	b	c	a	b
Sub-Patt =	a	b									
Delta$_2$ =										10	1

Fig 3.12

At $j = 7$ the sub-pattern, $\mathrm{Patt}[8,9] = \text{"ab"}$, recurs at index 5 and it is preceded by a different character 'd' ($\mathrm{Patt}[4]$), as shown below (in next page) in fig-3.13. So the sub-pattern recurrence is not preceded by the character 'c'. Its delta_2 value is,

delta₂ [7] = (7 + 1) – 5 + (9 – 7) = 5

Index =>	1	2	3	4	5	6	7	8	9
Patt =	a	b	c	d	a	b	c	a	b
Sub-Patt =				c	a	b			

Fig 3.13

Similarly the values of the remaining entries of delta_2 can be evaluated as follows,

delta₂ [6] = (6 + 1) – 0 + (9 – 6) = 10

Index =>	-1	0	1	2	3	4	5	6	7	8	9
Patt =	$	$	a	b	c	d	a	b	c	a	b
Sub-Patt =	b	c	a	b							

Fig 3.14

delta₂ [5] = (5 + 1) – (–1) + (9 – 5) = 11

Index =>	-2	-1	0	1	2	3	4	5	6	7	8	9
Patt =	$	$	$	a	b	c	d	a	b	c	a	b
Sub-Patt =	a	b	c	a	b							

Fig 3.15

delta₂ [4] = (4 + 1) – (–2) + (9 – 4) = 12

Index =>	-3	-2	-1	0	1	2	3	4	5	6	7	8	9
Patt =	$	$	$	$	a	b	c	d	a	b	c	a	b
Sub-Patt =	d	a	b	c	a	b							

Fig 3.16

Similarly the values of all the entries of the delta_2 table can be evaluated and the fully constructed delta_2 table is shown below in fig-3.17,

Delta₂ =	15	14	13	12	11	10	5	10	1
Index =>	1	2	3	4	5	6	7	8	9

Fig 3.17

4 Tables Usage With A Complete Example

This section demonstrates how Boyer-Moore algorithm usage delta_1 & delta_2 tables to find a pattern in a string. Let's reconsider the pattern **Patt** = **"abcdabcab"** and it's delta_2 table is already constructed in previous section (delta_2 Table sub-section), $\text{fig-}4.1$ below lists both of it's tables.

	a	b	c	d
Delta_1 =	1	0	2	5

Delta_2 =	15	14	13	12	11	10	5	10	1
Index =>	1	2	3	4	5	6	7	8	9

Fig 4.1

If string **S = "abcda-babeab-a-acabcdabcab"** then algorithm searches the pattern as follows,

i = 9

S =	a	b	c	d	a	-	b	a	b	e	a	b	-	a	-	a	c	a	b	c	d	a	b	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Patt =	a	b	c	d	a	b	c	a	b																	
Index =>	1	2	3	4	5	6	7	8	9																	

j = 9

Fig 4.2

As highlighted by the gray color filled cells above in $\text{fig-}4.2$ matching begins at indices $i = 9$ and $j = 9$. The last character of the pattern matches with the respective character of the string and the matching continues until index $j = 7$, where a mismatch occurs, as shown below in $\text{fig-}4.3$,

i = 7

S =	a	b	c	d	a	-	b	a	b	e	a	b	-	a	-	a	c	a	b	c	d	a	b	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Patt =	a	b	c	d	a	b	c	a	b																	
Index =>	1	2	3	4	5	6	7	8	9																	

j = 7

Fig 4.3

Now algorithm reads values from delta_1 and delta_2 tables. The mismatched character is 'b' (at $i = 7$), so it reads $\text{delta}_1['b']$, which is zero and mismatch occurs at $j = 7$ that's why reads $\text{delta}_2[7]$ which is five (tables shown above in $\text{fig-}4.1$). The maximum value is selected among them and i is updated to $i = i + \text{delta}_2[7] = 7 + 5 = 12$ and j is set to $j = \text{pattern length} = 9$, as shown below in $\text{fig-}4.4$.

Conceptually it means, pattern slides three positions to the right to align the recurrence of the sub-patter, $\text{Patt}[8, 9] = \text{"ab"}$, recurs at index 5 in the pattern and it is preceded by different character 'd' ($\text{Patt}[4]$), because rightmost occurrence of mismatched character 'b' is at the right side of the mismatch ($\text{Patt}[9]$) as shown above in $\text{fig-.}4.3$, so the value of delta_1 is irreverent here.

i = 12

S =	a	b	c	d	a	-	b	a	b	e	a	b	-	a	-	a	c	a	b	c	d	a	b	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Patt =				a	b	c	d	a	b	c	a	b														
Index =>				1	2	3	4	5	6	7	8	9														

j = 9

Fig 4.4

At $i = 12$ and $j = 9$ matching continues and after two matches, at $j = 7$ a mismatch occurs, as highlighted by the gray color filled cells below in $\text{fig-}4.5$,

i = 10

S =	a	b	c	d	a	-	b	a	b	e	a	b	-	a	-	a	c	a	b	c	d	a	b	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Patt =				a	b	c	d	a	b	c	a	b														
Index =>				1	2	3	4	5	6	7	8	9														

j = 7

Fig 4.5

The mismatch character 'e' (at $i = 10$) is not present in the patter and delta_1 table doesn't have an explicit entry $\text{delta}_1['e']$ rather it is assumed to has an implicit entry with pattern length as it's value ($\text{delta}_1['e'] = 9$) and for $j = 7$ the $\text{delta}_2[7]$ has value 5, as shown below in $\text{fig-}4.6$,

	a	b	c	d
Delta$_1$ =	1	0	2	5

Delta$_2$ =	15	14	13	12	11	10	5	10	1
Index =>	1	2	3	4	5	6	7	8	9

Fig 4.6

The max value among them is selected and i is updated to $i = i + \text{delta}_1['e'] = 10 + 9 = 19$ and as usual j is set to $j = \text{pattern length} = 9$, as shown below in $\text{fig-}4.7$.

Conceptually it means, patter slides seven positions to the right, beyond the character 'e' ($S[10]$).

i = 19

S =	a	b	c	d	a	-	b	a	b	e	a	b	-	a	-	a	c	a	b	c	d	a	b	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Patt =											a	b	c	d	a	b	c	a	b							
Index =>											1	2	3	4	5	6	7	8	9							

j = 9

Fig 4.7

At $i = 19$ and $j = 9$ matching continues and a mismatch occurs at $j = 6$, as shown below in $\text{fig-}4.8$,

i = 16

S =	a	b	c	d	a	-	b	a	b	e	a	b	-	a	-	a	c	a	b	c	d	a	b	c	a	b
Index =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Patt =											a	b	c	d	a	b	c	a	b							
Index =>											1	2	3	4	5	6	7	8	9							

j = 6

Fig 4.8

Index i is updated to $i = i + \max(\text{delta}_1['a'], \text{delta}_2[6]) = 16 + \max(1, 10) = 26$ and j set to $j = 9$, as highlighted by the gray color filled cells below (in next page) in $\text{fig-}4.9$,

What conceptually happens here is, the rightmost occurrence of mismatched character 'a' is at index 8 ($\text{Patt}[8]$) the right side of the mismatch position $j = 6$, as shown above in $\text{fig-}4.8$, but pattern can't slide left to align mismatched character, so value from delta_1 table is irreverent here. The pattern slides seven positions to the right, to align recurrence of sub-patter, $\text{Patt}[7, 8] = \text{"cab"}$, which recurs at index 0 in the pattern (how a sub-patter recurs beyond left-end of the pattern is already explained in the previous section).

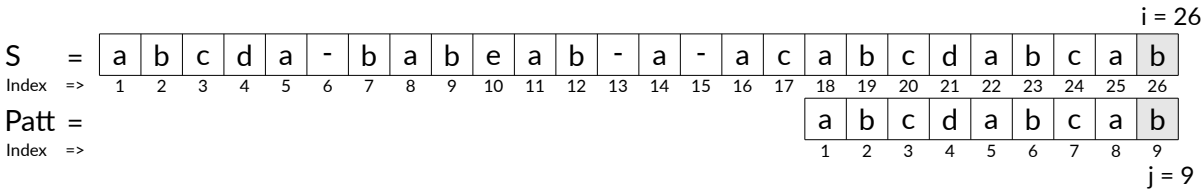

Fig 4.9

At $i = 26$ and $j = 9$ matching continues but this time no mismatch happens and algorithm finds the pattern in the string. Index j becomes zero as shown below in $\text{fig-}4.10$,

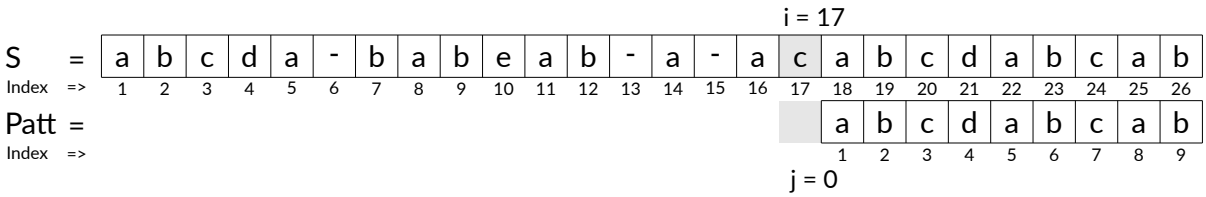

Fig 4.10

When j reaches to $j = 0$, it means all characters of the pattern are matched and algorithm returns $i + 1$ as a index where the patter is first found in the string, here at index $i + 1 = 17 + 1 = 18$, where the pattern is first found in the string.

5 Implementation

The $\mathrm{Boyer\text{-}Moore}$ algorithm and its variant the $\mathrm{Boyer\text{-}Moore\text{-}Horspool}$ algorithm are the part of $C++$ standard library since $C++17$. These are respectively `std::boyer_moore_searcher` and `std::boyer_moore_horspool_searcher`.

August 2026, India